FACULTÉ DES SCIENCES
DÉPARTEMENT D'INFORMATIQUE

UNIVERSITÉ LIBRE DE BRUXELLES ULB

INFO-F-203 – Algorithmique 2

Détection de communautés dans DBLP

2025-2026

Min-Tchun LI 000590125 Ayman Kahouache 000545130

1. Table des matières

Table des matières

1. Table des matières 1
2. Introduction 2
3. Méthodologie 2
   3.1. Tâche 1 2
      3.1.1. Construction de communautés 2
      3.1.2. Affichage des résultats 2
   3.2. Tâche 2 2
      3.2.1. Contrainte de traitement en flux 2
      3.2.2. Construction du graphe et de composantes fortement connexes 2
      3.2.3. Diamètre des 10 plus grandes communautés 3
         3.2.3.1. Choix d'algorithme du plus court chemin 3
      3.2.4. Affichage des résultats 3
   3.3. Tâche bonus 3
4. Résultats 3
   4.1. Tâche 1 3
   4.2. Tâche 2 3
   4.3. Tâche bonus 4
5. Discussion 4
   5.1. Résultat attendu 4
   5.2. Performance du script DBLPDemoParsing 4
   5.3. Efficacité de l'Union-Find 4
6. Conclusion 4
7. Figures, tableaux et annexes 5

2. Introduction

L'objectif est de pouvoir déterminer les différentes communautés de chercheur-euse-s à partir de la base de données bibliographique DBLP recensant les publications scientifiques en informatique.

L'analyse se basera selon deux critères :

- Les communautés de co-publication : groupes de personnes ayant une chaîne de co-publications reliant les auteur-rice-s.
- Les communautés fortes : groupes de chercheur-euse-s ayant au minimum 6 publications en commun.

3. Méthodologie

3.1. Tâche 1

3.1.1. Construction de communautés

Nous avons implémenté l'algorithme de la compression de chemins étant donné que l'algorithme est très efficace. En effet, d'après le théorème de Tarjan, pour M opérations faites sur N éléments à partir d'une structure Union-Find initialisée par la méthode union-rapide-pondérée avec compression de chemin, l'algorithme effectue $O(M\alpha(M, N))$ accès tableau, avec $\alpha(M, N)$, soit une complexité quasiment constante (proche de 1).

Pour pouvoir créer les communautés selon les critères, nous avons implémenté l'union rapide pondérée. On considère la structure comme un arbre, où chaque entrée du tableau id[i] désigne le parent de l'auteur i dans l'arborescence.

3.1.2. Affichage des résultats

Pour identifier les 10 plus grandes communautés, l'algorithme utilise un Min-Heap plutôt qu'un Max-Heap. Le principe est de comparer chaque nouvelle communauté à la racine (le plus petit élément du top 10). Si elle est plus grande, on remplace la racine. En utilisant un tas de taille fixe, l'espace utilisé est constant $O(1)$ alors que le Max-Heap dynamique nécessiterait un espace linéaire $O(n)$. Le traitement est plus rapide. En effet, on passe d'une complexité temporelle $n \log n$ pour le Max-Heap à $n \log 10$ pour le Min-Heap.

Pour gérer les auteurs en flux, une HashMap est utilisée, où la clé est le nom de l'auteur et la valeur est un identifiant unique. Sa complexité spatiale est de $O(n)$, où n est le nombre total d'auteurs distincts, ce qui permet une consommation mémoire proportionnelle à la progression de la recherche.

3.2. Tâche 2

3.2.1. Contrainte de traitement en flux

Afin de respecter la contrainte imposée, on incrémente un compteur associé à la paire (A, B) dans une structure `HashMap<String, HashMap<String, Integer>>` dont la clé externe est le nom de A et la clé interne le nom de B. Cela permet un accès et une mise à jour en temps amorti constant $O(1)$ par paire. Une fois le parsing terminé, on parcourt le HashMap pour ne conserver que les arêtes $A \to B$ dont le compteur est supérieur ou égal à 6 avant de construire le graphe orienté filtré.

3.2.2. Construction du graphe et de composantes fortement connexes

Pour la suite de la tâche, nous avons décidé d'utiliser la classe `Digraph` pour la construction de graphe dirigé, et `KosarajuSharirSCC` pour la construction de composantes fortement connexes, de `algs4`, librairie utilisée dans le code du syllabus.

3.2.3. Diamètre des 10 plus grandes communautés

Pour le calcul du plus court chemin, nous avons implémenté une fonction qui effectue un parcours en largeur (BFS). Bien que cette méthode garantisse l'exactitude du diamètre, sa complexité reste élevée pour des composantes de très grande taille.

En effet, comme un parcours BFS a une complexité temporelle $O(|E| + |V|)$ avec $|E|$ le nombre d'arêtes et $|V|$ le nombre de sommets, et qu'il faut exécuter le parcours sur tous les n sommets de la composante fortement connexe, l'algorithme a une complexité temporelle $O(n(|E| + |V|))$.

3.2.3.1. Choix d'algorithme du plus court chemin

Nous avons privilégié l'utilisation du parcours en largeur successifs plutôt que les algorithmes de Dijkstra ou de Bellman-Ford pour les raisons suivantes : la distance est définie comme le nombre d'arêtes du plus court chemin orienté. De ce fait, le graphe est considéré comme non-pondéré.

En conséquence, l'algorithme de Dijkstra se comporterait comme un BFS tout en gardant une surcharge de calcul inutile due à la gestion d'une file de priorité, augmentant la complexité de $O(V + E)$ à $O(E + V \log V)$.

En effet, Dijkstra est utilisé dans les graphes pondérés et Bellman-Ford est utilisé dans des graphes où les arêtes peuvent être pondérées négativement. Or dans la construction du graphe, les distances sont définies par le nombre d'arêtes, ce qui correspond à un graphe non-pondéré.

3.2.4. Affichage des résultats

De nouveau, pour l'affichage des résultats, nous avons utilisé la structure du Min-Heap qui permet d'avoir une bonne complexité temporelle ($O(n \log n)$) et une complexité spatiale constante ($O(1)$).

Par ailleurs, pour satisfaire la contrainte de traitement en flux, le comptage des paires ordonnées ($A \to B$) est effectué de manière online lors du parsing.

3.3. Tâche bonus

Nous avons utilisé les résultats de la tâche 2 via un fichier `.csv` contenant les 10 plus grandes communautés de co-publication.

Pour chaque auteur-rice, nous avons interrogé l'API OpenAlex afin d'identifier son institution et d'en déduire le pays associé. Les noms ont été préalablement nettoyés afin de supprimer les suffixes spécifiques à DBLP. En cas d'absence d'information, nous avons utilisé les affiliations comme source alternative. Les pays ont ensuite été agrégés par communauté afin de calculer le nombre d'auteur-rice-s par pays, puis convertis en proportions.

4. Résultats

4.1. Tâche 1

L'histogramme (Figure 2) révèle une distribution fortement asymétrique des communautés. La grande majorité des communautés sont très petites (entre 10 et 50 membres). On observe néanmoins l'existence d'une composante géante, visible à l'extrême droite de l'axe log regroupant à elle seule une fraction significative de l'ensemble des auteurs.

4.2. Tâche 2

L'histogramme Figure 3 confirme l'effet de filtrage à 6 publications communes : les communautés se contrent désormais entre 0 et 40 membres, sans composante géante dominante. La figure Figure 1 détaille les 10 plus grandes communautés obtenues. Les tailles varient de 21 à 133 auteurs, avec des

diamètres allant de 6 à 35. On note que la communauté la plus grande présente un diamètre de 30, tandis que la deuxième affiche le diamètre le plus élevé.

4.3. Tâche bonus

Le tableau généré (voir Figure 5) permet d'identifier si une communauté est associée à un ou plusieurs pays.

Le graphique « stacked bar chart » (voir Figure 6) met en évidence la proportion d'auteur-rice-s par pays pour chaque communauté.

5. Discussion

5.1. Résultat attendu

Les résultats obtenus sont cohérents avec les propriétés connues des réseaux de co-publication. La distribution en loi de puissance observée en tâche 1, avec une composante géante regroupant 92% des auteurs, confirme l'interconnexion forte de la communauté informatique mondiale. En tâche 2, le filtrage à 6 publications s'avère plus sévère qu'anticipé : le réseau se fragmente en communautés de 21 à 133 membres au maximum, suggérant que les collaborations profondes sont rares et localisées. Les diamètres élevés (jusqu'à 35 pour 73 membres) révèlent des structures allongées plutôt que densément connectées.

5.2. Performance du script DBLPDemoParsing

Le temps d'exécution total de de 1 min 47 (voir Figure 4) démontre l'efficience de l'approche en flux. Cette performance doit être mise en perspective avec le volume massif du snapshot de DBLP 2026. L'utilisation intensive du CPU démontre que la parallélisation du garbage collector de Java et l'efficacité de nos structures (Mini-Hep en $O(\log(10))$ et Union-Find) permettent de traiter le flux de manière fluide sans saturation mémoire. Cependant, il a été nécessaire d'ajouter un flag pour augmenter la taille du Heap pour la compilation pour éviter un crash lors de l'exécution.

Concernant le traitement de plus grandes bases de données, par exemple pour passer à l'échelle d'OpenAlex (480M de publications), la structure HashMap en RAM deviendrait un goulot d'étranglement. Une solution serait d'utiliser un stockage clé-valeur sur disque (comme RocksDB) ou de distribuer le graphe sur plusieurs serveurs. De plus, nous avions remarqué que certaines données étaient uniques, d'autres pas. Pour éviter de faire des calculs redondants, il serait judicieux d'utiliser un système de cache.

5.3. Efficacité de l'Union-Find

Le fait que le traitement de millions de publications ne prenne que 1 min 47 prouve que la compression de chemin et l'union pondérée maintiennent une complexité quasi-constante $\alpha(M, N)$ avec $\alpha(.)$ la fonction inverse d'Ackermann. Cela évite que le temps de calcul n'explose avec la croissance du graphe.

6. Conclusion

En conclusion, la tâche 1 a mis en évidence la structure caractéristique des réseaux de co-publication, dominés par une composante géante. La combinaison d'un filtrage strict pour la tâche 2 et d'un traitement en ligne pour la tâche 1 offre une vision complémentaire des structures de collaboration au sein de la communauté informatique mondiale. Ce qui pourrait être fait dans le futur pour optimiser le programme serait l'encodage des relations entre les auteurs par des identifiants plutôt que des chaînes de caractères car ces deriers prennent plus de place dans la mémoire.

7. Figures, tableaux et annexes

N°	Taille	Diamètre
1	133	30
2	73	35
3	38	13
4	34	7
5	33	6
6	30	17
7	27	10
8	26	12
9	23	6
10	21	7

Figure 1. – Tailles et diamètres des 10 plus grandes communautés (Tâche 2)

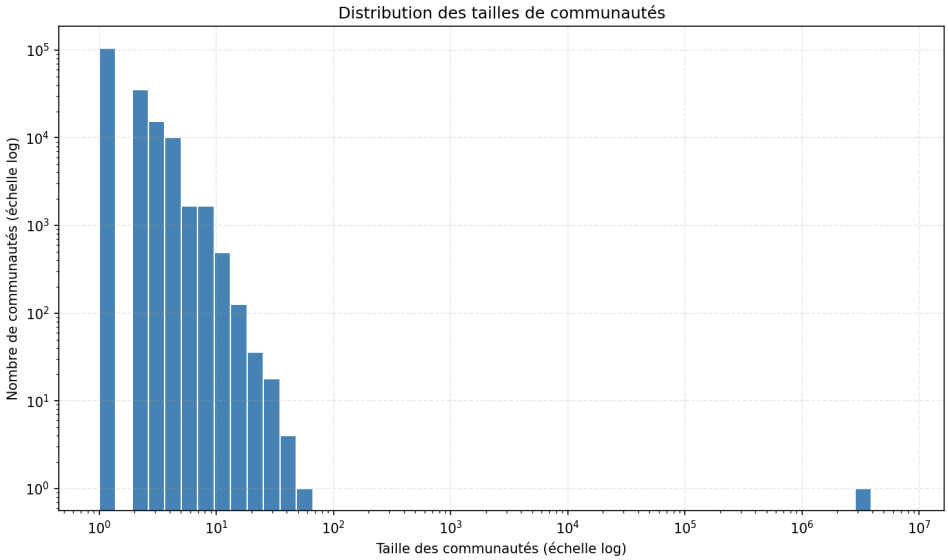

Figure 2. – Résultats du nombre de communautés, tâche 1.

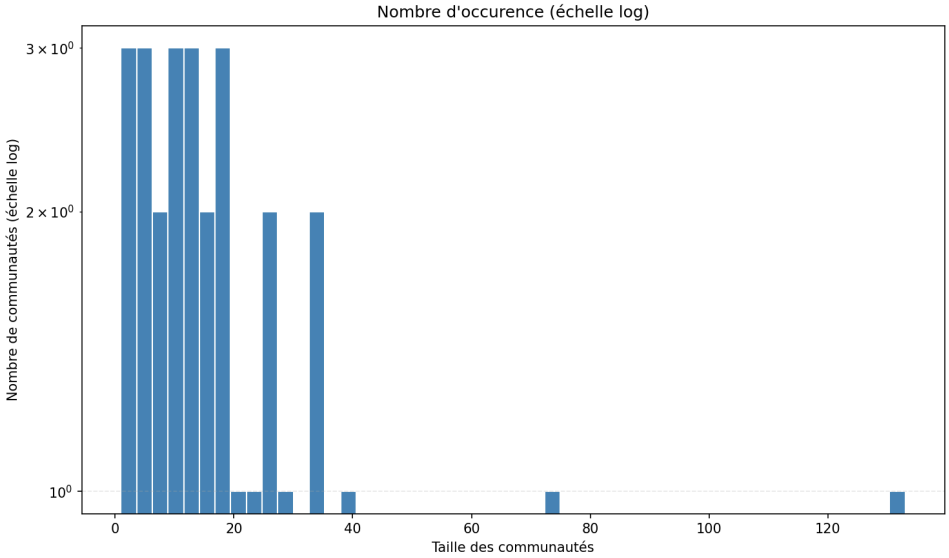

Figure 3. – Résultats du nombre de communautés, tâche 2.

```
└ time make > result.txt
make > result.txt  263,38s user 49,00s system 290% cpu 1:47,44 total
```

Figure 4. – Temps d'exécution et ressource matérielles.

Association des 10 plus grandes communautés à un ou plusieurs pays

Communautés	Liste de pays	Associations
1	SG, CN, US, CH, AU, GB, PH, NL, DE, IT, KR, HK, IN	Association multiple
2	GB, CN, CA, IE, US, ME, QA, SG, HK, DE	Association multiple
3	IT, AT, DE, US, CN, AU, FR	Association multiple
4	AL, JP, ES, US, IT, CN, PAYS INCONNU, PL	Association multiple
5	IT, US, BR, IE, GB, AL, DK, DE	Association multiple
6	CN, AU, RW, MO	Association multiple
7	CN, TW, PL, NO, BY, HK, US	Association multiple
8	FR, IT, GR, DK, CN, SE, US, ES, NL	Association multiple
9	CN, PAYS INCONNU, CH, GB, DE, IT, GR	Association multiple
10	CA, IT, FR, US, PAYS INCONNU, GB	Association multiple

Figure 5. – Association des communautés du top 10 à un ou plusieurs pays.

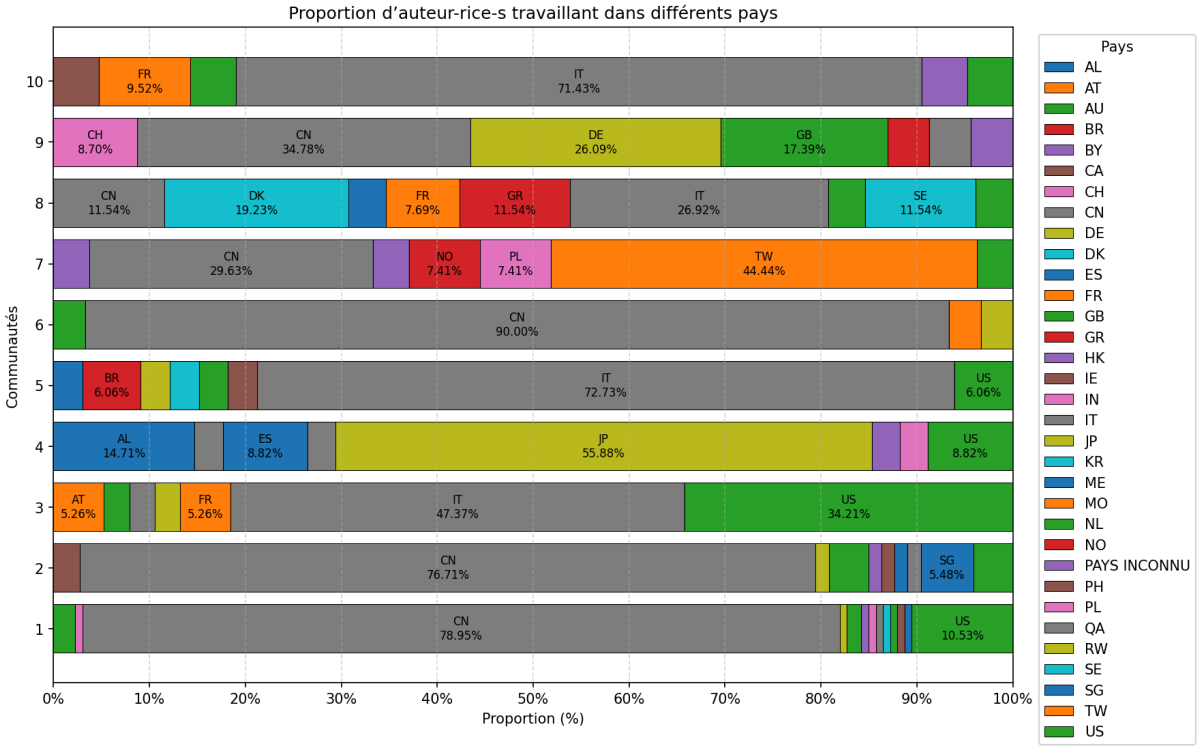

Figure 6. – Proportion d'auteur-rice-s travaillant dans différents pays au sein de chaque communauté du top 10.